

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 11 settembre 2009 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

COGNOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie (si noti che il punteggio pieno può essere ottenuto solo rispondendo alle parti non obbligatorie).
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. Si consideri il linguaggio regolare L delle stringhe di alfabeto $\{a, b\}$ che contengono sottostringhe di lunghezza due ripetute (eventualmente parzialmente sovrapposte).

Esempi:

$a a a$ $a b a a b$

Contresempi:

b $a a$ $a a b$ $a b b a a$

Si risponda alle domande seguenti:

- (a) Si definisca un automa nondeterministico che accetta il linguaggio L .
 - (b) Per entrambi il linguaggio L e per il suo complemento $\neg L$, si indichi se sono finiti o infiniti, giustificando la risposta.
-

Soluzione

2. È dato il linguaggio regolare L seguente.

$$L = ((a^* \mid b a) b)^+$$

Si risponda alle domande seguenti:

- (a) Si costruisca il riconoscitore deterministico del linguaggio L .
- (b) Si costruisca l'automa (deterministico o non) A_{suf} che riconosce il linguaggio regolare L_{suf} composto da tutti i suffissi propri delle frasi di L , precisamente definito come segue:

$$L_{suf} = \{ y \mid x y \in L \wedge |x| > 0 \}$$

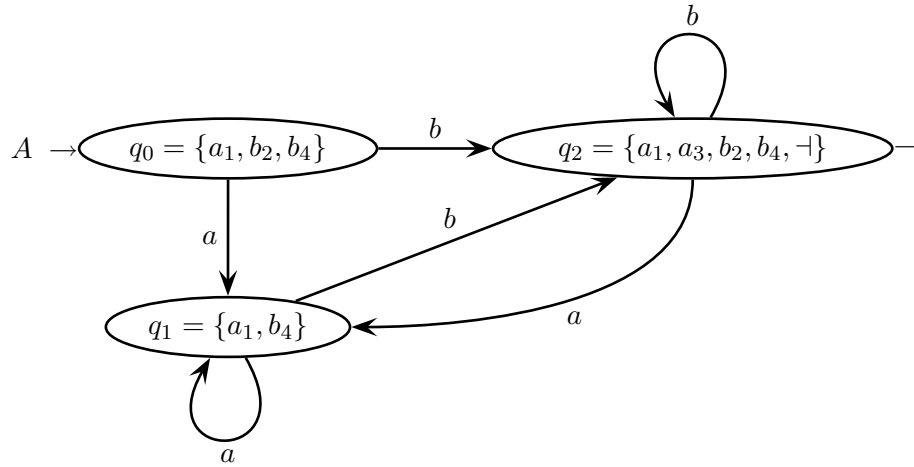
- (c) (facoltativa) Si descriva un algoritmo generale per risolvere il problema precedente nel caso di un linguaggio regolare L generico; ossia, dato un automa finito A , che si può supporre deterministico, si descriva un metodo per costruire l'automa che riconosce i suffissi propri L_{suf} del linguaggio $L = L(A)$.
-

Soluzione

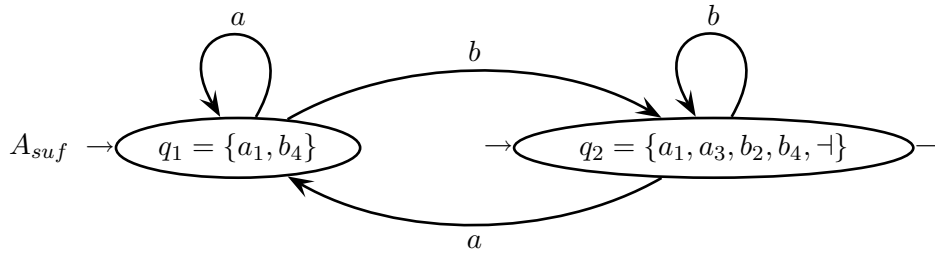
- (a) Occorre prima costruire il riconoscitore di L , mediante un metodo noto come quello di Berri-Sethi qui scelto. Dall'espressione regolare numerata

$$((a_1^* \mid b_2 a_3) b_4)^+$$

si ottiene l'automa A seguente:



Prima di riconoscere un suffisso proprio di una frase di L , l'automa A deve avere letti uno o più caratteri, ossia deve avere raggiunto lo stato q_1 oppure q_2 . L'automa A_{suf} si ottiene quindi semplicemente rendendo iniziali gli stati q_1 e q_2 , e togliendo l'attributo iniziale allo stato q_0 . Quest'ultimo diventa allora inutile e va cancellato.



- (b) L'approccio seguito nell'esercizio precedente risulta valido soltanto quando lo stato iniziale di A non contiene transizioni entranti, che permetterebbero alla macchina di riconoscere un prefisso di frase, pur restando nello stato iniziale, così provocando la perdita di alcuni suffissi.

Una costruzione valida in generale è la seguente, che sfrutta la proprietà di chiusura dei linguaggi regolari rispetto alla riflessione.

- Costruisci il riconoscitore deterministico A_R del linguaggio riflesso L^R . Si osserva che i prefissi propri di L^R coincidono con i suffissi propri di L .
- Trasforma A_R nella macchina M così ottenuta:
 - rendi finale ogni stato non finale di A_R
 - mantieni come finale ogni stato finale di A_R da cui escano transizioni
 - rendi non finale ogni altro stato finale di A_R , stato che così diviene inutile
- Costruisci l'automa che riconosce il linguaggio speculare $(L(M))^R$. Esso è l'automa A_{suf} richiesto.

2 Grammatiche libere e automi a pila 20%

1. È dato il linguaggio libero L seguente, di alfabeto $\{a, b\}$:

$$L = \{ a^n b^n \mid n \geq 0 \} \cup \{ a^{2h} b^{2k} \mid h \geq k \geq 0 \}$$

Si risponda alle domande seguenti:

- (a) Si progetti una grammatica G che genera il linguaggio L , ambigua o no.
 - (b) (facoltativa) Si argomenti se la grammatica G progettata è ambigua oppure no, e nel caso G sia ambigua, se ne progetti una equivalente non ambigua.
-

Soluzione

- (a) È rapido scrivere la grammatca G unione delle due componenti del linguaggio L , entrambe piuttosto standard (assioma S).

$$G \left\{ \begin{array}{l} S \rightarrow X \mid Y \\ X \rightarrow a X b \mid \varepsilon \\ Y \rightarrow a a Y b b \mid A \\ A \rightarrow a a A \mid \varepsilon \end{array} \right.$$

- (b) Si osservi che i due componenti dell'unione che definisce il linguaggio L hanno in comune le stringhe $a^{2n} b^{2n}$ ($n \geq 0$). Pertanto G , costruita su tale modello, è ambigua e le stringhe ambigue (di grado due) sono proprio $a^{2n} b^{2n}$ ($n \geq 0$).

Per ottenere una grammatica non ambigua, basta raffinare la scomposizione del linguaggio L come unione, eliminando le stringhe ambigue da una delle due componenti. Per esempio:

$$L = \{ a^n b^n \mid n \geq 0 \} \cup \{ a^{2h} b^{2k} \mid h > k \geq 0 \}$$

oppure

$$L = \{ a^{2n+1} b^{2n+1} \mid n \geq 0 \} \cup \{ a^{2h} b^{2k} \mid h \geq k \geq 0 \}$$

Poi basta prendere una delle due formulazioni e costruire la grammatica unione delle due componenti, ancora piuttosto standard. Eccole entrambe:

$$G \left\{ \begin{array}{l} S \rightarrow X \mid Y \\ X \rightarrow a X b \mid \varepsilon \\ Y \rightarrow a a Y b b \mid A' \\ A' \rightarrow a a A' \mid a a \end{array} \right. \quad G \left\{ \begin{array}{l} S \rightarrow X' \mid Y \\ X' \rightarrow a a X' b b \mid a b \\ Y \rightarrow a a Y b b \mid A \\ A \rightarrow a a A \mid \varepsilon \end{array} \right.$$

L'apice e il colore rosso mettono in evidenza i nonterminali e le parti di regola ritoccati rispetto alla formualzione ambigua.

2. Si deve definire la sintassi della dichiarazione di alcuni tipi di tabella nello stile del linguaggio Java. Vanno trattati i casi seguenti:

- la tabella può avere una o più dimensioni
- gli elementi della tabella sono di tipo intero
- i valori iniziali degli elementi della tabella possono essere specificati mediante costanti o espressioni costanti (che non possono contenere parentesi)
- il comando **new** è presente quando necessario

Gli esempi seguenti mostrano alcuni casi; naturalmente sono possibili altre combinazioni, e vanno trattate.

```
int [ ] oneDimArray = new int [42] ;  
int [ ] even = new int [ ] {2, 4, 6, 2 * 5} ;  
int [ ] odd = {3, 5, 3 + 4, 3 + 4 + 2} ;  
int [ ] even2 = new int [4] {2, 4, 6, 2 * 5} ;  
int [ ] [ ] tableau = new int [10] [10] ;
```

Gli identificatori degli oggetti sono codificati dal terminale **id** e i numeri interi dal terminale **num** (né **id** né **num** vanno espansi ulteriormente).

Si risponda alle domande seguenti:

- (a) Si scriva la grammatica che genera una o più dichiarazioni. Si usi la forma estesa EBNF, evitando le ambiguità.
- (b) Si discuta anche il caso dell'inizializzazione di array a due o più dimensioni. Il caso è illustrato dagli esempi seguenti:

```
int [ ] [ ] twoDimArray = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} } ;  
int [ ] [ ] myArray = new int [5] [ ] ;
```

Soluzione

3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica G seguente (assioma E):

$$G \left\{ \begin{array}{l} E \rightarrow ' (' T ' : ' T ') ' \mid ' (' T ' : ' T ' : ' T ') ' \\ T \rightarrow i ' : ' i \mid i ' \times ' i \mid i \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si disegnino gli automi corrispondenti alle regole della grammatica G e, mostrando su di essi gli insiemi guida rilevanti di lunghezza $k = 1$, si dica se la grammatica è $LL(1)$.
 - (b) In caso negativo, considerando gli insiemi guida rilevanti di lunghezza $k = 2$, si dica se la grammatica G è $LL(2)$.
 - (c) (facoltativa) In caso negativo, si discuta come ottenere una grammatica G' equivalente a G e adatta all'analisi $LL(k)$, per un $k \geq 1$.
-

Soluzione

2. È data la grammatica G seguente, di alfabeto $\{a, b\}$ (assioma S):

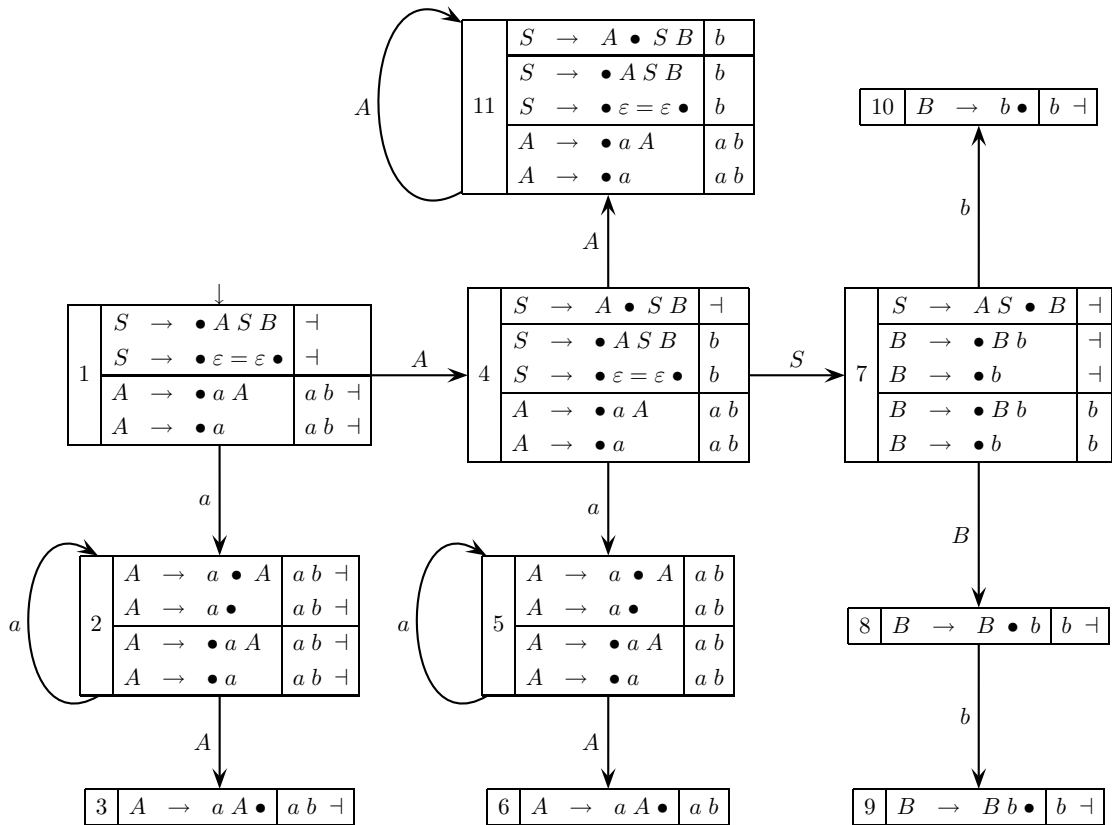
$$G \left\{ \begin{array}{l} S \rightarrow A S B \\ S \rightarrow \varepsilon \\ A \rightarrow a A \mid a \\ B \rightarrow B b \mid b \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si argomenti se la grammatica G è ambigua oppure no, motivando la risposta.
 - (b) Qualunque sia l'esito del punto (a), si tracci il grafo pilota di tipo $LR(1)$ dell'analizzatore sintattico associato alla grammatica G , fino a individuare un primo eventuale stato inadeguato, indicando il tipo di conflitto di cui soffre.
-

Soluzione

- (a) È facile osservare che la grammatica G è fortemente ambigua: quasi tutte le stringhe sono generate ambigualmente, con grado di ambiguità crescente. Per esempio, $a^n b^n$ con $n \geq 2$ è ambigua.
- (b) Ecco il grafo pilota $LR(1)$, che ovviamente contiene stati inadeguati, essendo la grammatica G ambigua:



Il grafo pilota ha 15 macrostati. Dal macrostato 11 esce un arco a diretto al macrostato 5, e sempre a 11 sono collegati quattro macrostati identici a 7, 8, 9 e 10, tranne togliere il terminatore $\vdash$ dalla prospezione, qui non disegnati.

I macrostati 2 e 5 sono inadeguati, per conflitto riduzione-spostamento sul terminale a . Non ci sono altri conflitti.

Si noti che il linguaggio L in realtà è regolare: $L = \varepsilon \mid a^+ b^+$, dunque non è inerentemente ambiguo e anzi è $LR(1)$, sebbene non $LR(0)$ perché ha prefissi.

4 Traduzione e analisi semantica 20%

1. Si consideri il linguaggio regolare L seguente:

$$L = (0 \mid 1)^* 0 (0 \mid 1)^*$$

Esso è composto da stringhe di zeri e uni contenenti almeno uno zero. Si interpretino le stringhe di L come rappresentazioni binarie di numeri naturali. Per esempio, la stringa 00100101 rappresenta il numero naturale 37.

$$00100101 = 2^5 + 2^2 + 2^0 = 32 + 4 + 1 = 37$$

Per tale linguaggio L si vuole codificare la funzione di traduzione τ che a ogni stringa s che rappresenta un numero naturale n , fa corrispondere la stringa che rappresenta il numero naturale $n + 1$.

Esempio:

$$\tau(00100101) = 00100110$$

Si risponda alle domande seguenti:

- (a) Si codifichi la funzione di traduzione τ mediante una trasduzione deterministica a due passate riflesse.
 - (b) (facoltativa) Si codifichi la relazione di traduzione corrispondente a τ , mediante un automa a due ingressi.
-

Soluzione

2. È dato il sottoprogramma seguente:

```
int  $f$  ( int  $x$ ,  $z$  ) {  
    int  $y$ ,  $i$ ;  
     $y = x * x - 10$ ;  
    for (  $i = 1$ ;  $i \leq 100$ ;  $i++$  ) {  
         $y = y * 2$ ;  
    } /* for */  
    return ( $z + y$ );  
} /* f */
```

Si risponda alle domande seguenti:

- (a) Si disegni il grafo di controllo, numerando progressivamente 1, 2, ... le istruzioni come riferimento. Si scrivano le equazioni di flusso per calcolare le variabili vive in ogni punto.
 - (b) Si calcoli la soluzione del sistema di equazioni.
 - (c) (facoltativa) Si dica quante celle di memoria sono necessarie per le variabili x , z , i , y e f .
-

Soluzione

